

Project

dataset link:

Relationships:

- `customers` ↔ `accounts` (1:N)
 - `customers` ↔ `loans` (1:N)
 - `accounts` ↔ `transactions` (1:N)
 - `branches` ↔ `transactions` (1:N)
 - Reference tables provide valid values for types/statuses.
-

Data Quality Issues (intentionally added)

To simulate real-world scenarios, ~7% of the data includes errors:

- 2% missing values (nulls)
 - 1% typos in text fields
 - 1% duplicate rows
 - 1% inconsistent number/text formats
 - 1% mixed date formats
 - 1% non-sequential IDs
 - 1% future dates
-

BASIC LEVEL (Questions 1-10)

Focus: Single-table queries, filtering, sorting, basic aggregations

Q1. List all customers with their basic information

```
-- Show customer ID, type, date of birth, and address ID for
the first 20 customers
SELECT customer_id, customer_type_id, date_of_birth, address_
id
FROM customers
LIMIT 20;
```

Q2. Find all accounts with balance greater than \$10,000

```
SELECT account_id, customer_id, account_type_id, balance, acc
ount_status_id
FROM accounts
WHERE balance > 10000
ORDER BY balance DESC;
```

Q3. Count total number of transactions in the dataset

```
SELECT COUNT(*) AS total_transactions
FROM transactions;
```

Q4. Find all loans that are in default status

```
SELECT loan_id, customer_id, loan_type_id, loan_amount, inter
est_rate, loan_status_id
FROM loans
WHERE loan_status_id = 4; -- Assuming 4 = defaulted
```

Q5. List all fraudulent transactions with amount greater than \$5,000

```
SELECT transaction_id, account_id, amount, transaction_dateti
me, is_fraud
FROM transactions
```

```
WHERE is_fraud = TRUE AND amount > 5000  
ORDER BY amount DESC;
```

Q6. Show unique transaction types available

```
SELECT DISTINCT transaction_type_id  
FROM transactions  
ORDER BY transaction_type_id;
```

Q7. Find customers born after January 1, 1990

```
SELECT customer_id, date_of_birth  
FROM customers  
WHERE date_of_birth > '1990-01-01'  
ORDER BY date_of_birth DESC;
```

Q8. Count how many accounts are currently active vs closed

```
SELECT account_status_id, COUNT(*) AS account_count  
FROM accounts  
GROUP BY account_status_id;
```

Q9. Find the top 10 highest-value transactions that are NOT fraudulent

```
SELECT transaction_id, account_id, amount, transaction_datetime  
FROM transactions  
WHERE is_fraud = FALSE  
ORDER BY amount DESC  
LIMIT 10;
```

Q10. List all branches located in a specific city (e.g., 'New York')

```
SELECT branch_id, branch_name, location
FROM branches
WHERE location ILIKE '%New York%'; -- ILIKE for case-insensitive search
```

🟡 INTERMEDIATE LEVEL (Questions 11-20)

Focus: Joins, aggregations, GROUP BY, HAVING, subqueries

Q11. Show each customer's full name (from customers) and their total account balance

```
SELECT c.customer_id,
       SUM(a.balance) AS total_balance
FROM customers c
INNER JOIN accounts a ON c.customer_id = a.customer_id
GROUP BY c.customer_id
ORDER BY total_balance DESC;
```

Q12. Find customers who have both loans AND accounts

```
SELECT DISTINCT c.customer_id
FROM customers c
INNER JOIN accounts a ON c.customer_id = a.customer_id
INNER JOIN loans l ON c.customer_id = l.customer_id;
```

Q13. Calculate average loan amount by loan status

```
SELECT ls.status_name,
       AVG(l.loan_amount) AS avg_loan_amount,
       COUNT(*) AS num_loans
FROM loans l
INNER JOIN loan_statuses ls ON l.loan_status_id = ls.loan_status_id;
```

```
tus_id  
GROUP BY ls.status_name  
ORDER BY avg_loan_amount DESC;
```

Q14. Find branches with total transaction volume exceeding \$1,000,000

```
SELECT b.branch_name,  
       SUM(t.amount) AS total_volume  
FROM branches b  
INNER JOIN transactions t ON b.branch_id = t.branch_id  
WHERE t.is_fraud = FALSE  
GROUP BY b.branch_name  
HAVING SUM(t.amount) > 1000000  
ORDER BY total_volume DESC;
```

Q15. Identify customers who have made more than 50 transactions

```
SELECT c.customer_id,  
       COUNT(t.transaction_id) AS transaction_count  
FROM customers c  
INNER JOIN accounts a ON c.customer_id = a.customer_id  
INNER JOIN transactions t ON a.account_id = t.account_id  
GROUP BY c.customer_id  
HAVING COUNT(t.transaction_id) > 50  
ORDER BY transaction_count DESC;
```

Q16. Show account details along with customer type (personal/corporate)

```
SELECT a.account_id,  
       a.balance,  
       c.customer_id,
```

```

        ct.type_name AS customer_type
FROM accounts a
INNER JOIN customers c ON a.customer_id = c.customer_id
INNER JOIN customer_types ct ON c.customer_type_id = ct.customer_type_id;

```

Q17. Find the most common transaction type overall

```

SELECT tt.type_name,
       COUNT(*) AS usage_count
FROM transactions t
INNER JOIN transaction_types tt ON t.transaction_type_id = tt.transaction_type_id
GROUP BY tt.type_name
ORDER BY usage_count DESC
LIMIT 1;

```

Q18. Calculate total fraudulent amount per account

```

SELECT account_id,
       SUM(amount) AS total_fraud_amount,
       COUNT(*) AS fraud_transaction_count
FROM transactions
WHERE is_fraud = TRUE
GROUP BY account_id
ORDER BY total_fraud_amount DESC;

```

Q19. Find customers with no associated accounts (orphaned records)

```

SELECT c.customer_id
FROM customers c

```

```
LEFT JOIN accounts a ON c.customer_id = a.customer_id
WHERE a.account_id IS NULL;
```

Q20. List all transactions with their branch name and transaction type

```
SELECT t.transaction_id,
       t.amount,
       b.branch_name,
       tt.type_name AS transaction_type,
       t.is_fraud
FROM transactions t
LEFT JOIN branches b ON t.branch_id = b.branch_id
LEFT JOIN transaction_types tt ON t.transaction_type_id = tt.transaction_type_id
LIMIT 100;
```

ADVANCED LEVEL (Questions 21-25)

Focus: Window functions, CTEs, date functions, case logic

Q21. Rank customers by total spending within each customer type (personal/corporate)

```
WITH customer_spending AS (
    SELECT c.customer_id,
           ct.type_name,
           SUM(t.amount) AS total_spent
    FROM customers c
    INNER JOIN customer_types ct ON c.customer_type_id = ct.customer_type_id
    INNER JOIN accounts a ON c.customer_id = a.customer_id
    INNER JOIN transactions t ON a.account_id = t.account_id
    WHERE t.is_fraud = FALSE
    GROUP BY c.customer_id, ct.type_name
```

```

)
SELECT customer_id,
       type_name,
       total_spent,
       RANK() OVER (PARTITION BY type_name ORDER BY total_spent DESC) AS rank_in_type
FROM customer_spending;

```

Q22. Calculate month-over-month growth in transaction volume

```

WITH monthly_volume AS (
    SELECT DATE_TRUNC('month', transaction_datetime) AS month,
           SUM(amount) AS total_volume
    FROM transactions
    WHERE is_fraud = FALSE
    GROUP BY DATE_TRUNC('month', transaction_datetime)
)
SELECT month,
       total_volume,
       LAG(total_volume) OVER (ORDER BY month) AS previous_month_volume,
       ROUND(100.0 * (total_volume - LAG(total_volume) OVER
                                (ORDER BY month))
              / LAG(total_volume) OVER (ORDER BY month), 2) AS percent_growth
FROM monthly_volume
ORDER BY month;

```

Q23. Find suspicious accounts: high average transaction amount but low balance

```

SELECT a.account_id,
       a.balance,
       AVG(t.amount) AS avg_transaction_amount,

```

```

COUNT(t.transaction_id) AS txn_count
FROM accounts a
INNER JOIN transactions t ON a.account_id = t.account_id
WHERE t.is_fraud = FALSE
GROUP BY a.account_id, a.balance
HAVING AVG(t.amount) > a.balance * 0.5 -- Avg transaction >
50% of balance
AND COUNT(t.transaction_id) > 10
ORDER BY avg_transaction_amount DESC;

```

Q24. For each customer, show their loan payment performance (if loan amount > total transactions)

```

SELECT c.customer_id,
       l.loan_amount,
       COALESCE(SUM(t.amount), 0) AS total_transactions,
       CASE
           WHEN COALESCE(SUM(t.amount), 0) > l.loan_amount THEN
EN 'Good - Exceeds Loan'
           WHEN COALESCE(SUM(t.amount), 0) > l.loan_amount *
0.5 THEN 'Moderate - Over 50%'
           WHEN COALESCE(SUM(t.amount), 0) > 0 THEN 'Poor - U
nder 50%'
           ELSE 'No Activity - High Risk'
       END AS payment_risk_category
FROM customers c
LEFT JOIN loans l ON c.customer_id = l.customer_id
LEFT JOIN accounts a ON c.customer_id = a.customer_id
LEFT JOIN transactions t ON a.account_id = t.account_id
WHERE l.loan_id IS NOT NULL
GROUP BY c.customer_id, l.loan_amount;

```

Q25. Detect potential money laundering: multiple transactions just below reporting threshold

```

SELECT account_id,
       COUNT(*) AS transaction_count,
       SUM(amount) AS total_amount,
       AVG(amount) AS avg_amount,
       MAX(amount) AS max_amount
FROM transactions
WHERE amount BETWEEN 9000 AND 9999 -- Just below $10,000 reporting threshold
      AND is_fraud = FALSE
GROUP BY account_id
HAVING COUNT(*) > 5 -- More than 5 near-threshold transactions
ORDER BY transaction_count DESC;

```

EXPERT LEVEL (Questions 26-30)

Focus: Query optimization, recursive CTEs, stored procedures, complex business logic

Q26. Find the longest sequence of transactions for each account without any fraud

```

WITH fraud_flags AS (
    SELECT account_id,
           transaction_id,
           transaction_datetime,
           is_fraud,
           ROW_NUMBER() OVER (PARTITION BY account_id ORDER BY transaction_datetime) -
           ROW_NUMBER() OVER (PARTITION BY account_id, is_fraud ORDER BY transaction_datetime) AS grp
    FROM transactions
),
fraud_free_sequences AS (

```

```

SELECT account_id,
       COUNT(*) AS clean_sequence_length,
       MIN(transaction_datetime) AS sequence_start,
       MAX(transaction_datetime) AS sequence_end
FROM fraud_flags
WHERE is_fraud = FALSE
GROUP BY account_id, grp
)
SELECT account_id,
       MAX(clean_sequence_length) AS longest_fraud_free_streak
FROM fraud_free_sequences
GROUP BY account_id
ORDER BY longest_fraud_free_streak DESC
LIMIT 10;

```

Q27. Reconcile accounts: Find mismatches between transaction history and stated balance

```

WITH transaction_balance AS (
  SELECT a.account_id,
         a.balance AS stated_balance,
         SUM(CASE
              WHEN tt.type_name IN ('deposit', 'payment') THEN
                t.amount
              WHEN tt.type_name IN ('withdrawal', 'transfer') THEN -t.amount
              ELSE 0
            END) AS calculated_balance_from_txns
  FROM accounts a
  INNER JOIN transactions t ON a.account_id = t.account_id
  INNER JOIN transaction_types tt ON t.transaction_type_id = tt.transaction_type_id
  WHERE t.is_fraud = FALSE
  GROUP BY a.account_id, a.balance
)

```

```

)
SELECT account_id,
       stated_balance,
       calculated_balance_from_txns,
       stated_balance - calculated_balance_from_txns AS discrepancy,
       ABS(stated_balance - calculated_balance_from_txns) / NULLIF(stated_balance, 0) * 100 AS discrepancy_percent
FROM transaction_balance
WHERE ABS(stated_balance - calculated_balance_from_txns) > 0.01
ORDER BY discrepancy_percent DESC;

```

Q28. Build a customer 360 view with risk scoring (using multiple CTEs)

```

WITH customer_activity AS (
    SELECT c.customer_id,
           COUNT(DISTINCT a.account_id) AS num_accounts,
           COUNT(DISTINCT l.loan_id) AS num_loans,
           COUNT(t.transaction_id) AS total_transactions,
           SUM(t.amount) AS total_spent,
           SUM(CASE WHEN t.is_fraud THEN 1 ELSE 0 END) AS fraud_transactions
    FROM customers c
    LEFT JOIN accounts a ON c.customer_id = a.customer_id
    LEFT JOIN loans l ON c.customer_id = l.customer_id
    LEFT JOIN transactions t ON a.account_id = t.account_id
    GROUP BY c.customer_id
),
risk_scores AS (
    SELECT customer_id,
           CASE WHEN total_spent > 100000 THEN 'High Value' ELSE 'Standard' END AS value_tier,
           CASE WHEN fraud_transactions > 0 THEN 'Compromise

```

```

d'
        WHEN num_loans > 3 THEN 'High Debt'
        ELSE 'Low Risk'
    END AS risk_tier,
    ROUND(100.0 * fraud_transactions / NULLIF(total_transactions, 0), 2) AS fraud_rate_percent
FROM customer_activity
)
SELECT ca.*,
       rs.value_tier,
       rs.risk_tier,
       rs.fraud_rate_percent
FROM customer_activity ca
INNER JOIN risk_scores rs ON ca.customer_id = rs.customer_id
WHERE rs.risk_tier != 'Low Risk'
ORDER BY rs.fraud_rate_percent DESC;

```

Q29. Optimize a slow query with proper indexing and execution plan analysis

```

-- PROBLEM: Slow query that finds customers with defaulted loans AND fraudulent transactions
-- Before optimization (likely full table scans)

-- Step 1: Create strategic indexes
CREATE INDEX idx_loans_customer_status ON loans(customer_id, loan_status_id);
CREATE INDEX idx_transactions_account_fraud ON transactions(account_id, is_fraud);
CREATE INDEX idx_accounts_customer ON accounts(customer_id);
CREATE INDEX idx_transactions_datetime ON transactions(transaction_datetime);

-- Step 2: Rewrite query using EXISTS instead of DISTINCT JOINS

```

```

SELECT c.customer_id,
       c.date_of_birth,
       (SELECT COUNT(*) FROM loans l WHERE l.customer_id = c.
customer_id AND l.loan_status_id = 4) AS defaulted_loans_coun
t,
       (SELECT COUNT(*) FROM accounts a
INNER JOIN transactions t ON a.account_id = t.account
_id
       WHERE a.customer_id = c.customer_id AND t.is_fraud =
TRUE) AS fraud_txn_count
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM loans l
    WHERE l.customer_id = c.customer_id AND l.loan_status_id
= 4
)
AND EXISTS (
    SELECT 1 FROM accounts a
    INNER JOIN transactions t ON a.account_id = t.account_id
    WHERE a.customer_id = c.customer_id AND t.is_fraud = TRUE
);

```

-- Step 3: Analyze query plan

```
EXPLAIN ANALYZE
```

```

SELECT c.customer_id, c.date_of_birth
FROM customers c
WHERE EXISTS (SELECT 1 FROM loans l WHERE l.customer_id = c.c
ustomer_id AND l.loan_status_id = 4)
AND EXISTS (SELECT 1 FROM accounts a INNER JOIN transactions
t ON a.account_id = t.account_id WHERE a.customer_id = c.cust
omer_id AND t.is_fraud = TRUE);

```

Q30. Create a stored procedure for automated fraud detection alerts

```

-- Stored procedure to flag suspicious transactions and insert
t into alert table
CREATE OR REPLACE PROCEDURE detect_fraud_alerts(p_days_back I
INTEGER DEFAULT 1)
LANGUAGE plpgsql
AS $$
DECLARE
    v_alert_count INTEGER := 0;
BEGIN
    -- Create alerts table if it doesn't exist
    CREATE TABLE IF NOT EXISTS fraud_alerts (
        alert_id SERIAL PRIMARY KEY,
        transaction_id INTEGER,
        account_id INTEGER,
        alert_reason TEXT,
        alert_score INTEGER,
        detected_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        reviewed BOOLEAN DEFAULT FALSE
    );

    -- Insert high-risk transactions into alerts
    INSERT INTO fraud_alerts (transaction_id, account_id, ale
rt_reason, alert_score)
    SELECT
        t.transaction_id,
        t.account_id,
        CASE
            WHEN t.amount > 10000 AND t.transaction_datetime
> CURRENT_TIME - INTERVAL '1 hour' THEN 'High value after hou
rs'
            WHEN a.balance < t.amount * 2 AND t.is_fraud = FA
LSE THEN 'Near overdraft with large transaction'
            WHEN EXISTS (SELECT 1 FROM transactions t2 WHERE
t2.account_id = t.account_id AND t2.is_fraud = TRUE
                        AND t2.transaction_datetime > t.tran

```

```

saction_datetime - INTERVAL '1 day') THEN 'Recent fraud on ac
count'

        ELSE 'Unusual pattern detected'
    END AS alert_reason,
    CASE
        WHEN t.amount > 10000 THEN 90
        WHEN a.balance < t.amount * 2 THEN 70
        WHEN EXISTS (SELECT 1 FROM transactions t2 WHERE
t2.account_id = t.account_id AND t2.is_fraud = TRUE) THEN 95
        ELSE 50
    END AS alert_score
FROM transactions t
INNER JOIN accounts a ON t.account_id = a.account_id
WHERE t.transaction_datetime >= CURRENT_DATE - (p_days_ba
ck || ' days')::INTERVAL
    AND t.is_fraud = FALSE -- Not already flagged as fraud
    AND (
        t.amount > 10000
        OR a.balance < t.amount * 2
        OR EXISTS (SELECT 1 FROM transactions t2 WHERE t2.a
ccount_id = t.account_id AND t2.is_fraud = TRUE
                    AND t2.transaction_datetime > t.transact
ion_datetime - INTERVAL '1 day')
    )
    AND NOT EXISTS (SELECT 1 FROM fraud_alerts fa WHERE fa.
transaction_id = t.transaction_id); -- Avoid duplicates

GET DIAGNOSTICS v_alert_count = ROW_COUNT;
RAISE NOTICE 'Detected % potential fraud alerts for the l
ast % days', v_alert_count, p_days_back;
END;
$$;

-- Execute the procedure
CALL detect_fraud_alerts(7);

```

```
-- Query alerts
SELECT * FROM fraud_alerts WHERE reviewed = FALSE ORDER BY alert_score DESC;
```

Quick Reference Table for Interview Prep

Question	Primary Skills	Difficulty	Interview Likelihood
Q1-3	Basic SELECT, LIMIT	Basic	100% (warm-up)
Q4-6	WHERE, ORDER BY, COUNT	Basic	90%
Q7-10	DISTINCT, GROUP BY, ILIKE	Basic-Intermediate	85%
Q11-13	INNER JOIN, SUM, AVG	Intermediate	95%
Q14-16	HAVING, multiple joins	Intermediate	90%
Q17-20	LEFT JOIN, aggregation	Intermediate	85%
Q21-22	Window functions, CTEs	Advanced	80%
Q23-25	CASE, date functions	Advanced	75%
Q26-27	Recursive logic, reconciliation	Expert	60%
Q28-30	Multi-CTE, optimization, procedures	Expert	50% (senior roles)

Portfolio Submission Checklist

For each question, include in your GitHub repository:

1. **SQL file** with the query (well-commented)
2. **Screenshot** of results (first 10 rows)
3. **Markdown explanation** including:
 - Business question being answered
 - Skills demonstrated
 - Query logic explanation

- Any optimizations applied
- Sample insight from results

4. Bonus points for:

- Creating views for frequently used queries
- Building a dashboard in Tableau/Power BI connected to these queries
- Writing unit tests for critical business logic
- Documenting query performance metrics

Would you like me to also provide the `CREATE TABLE` statements and data import script to set up this entire database locally? This would allow you to test and run all 30 queries yourself.

Complete SQL Solutions for Finance Fraud & Loans Dataset

Section 1: Data Quality & Integrity (Foundational)

1. Null Value Analysis Across All Tables

```
-- Query to count NULL values in every column across all tables
-- Run this for each table individually

-- For customers table
SELECT
    'customers' AS table_name,
    COUNT(*) - COUNT(customer_id) AS null_customer_id,
    COUNT(*) - COUNT(customer_type_id) AS null_customer_type_id,
    COUNT(*) - COUNT(address_id) AS null_address_id,
    COUNT(*) - COUNT(date_of_birth) AS null_date_of_birth,
    COUNT(*) - COUNT(first_name) AS null_first_name,
    COUNT(*) - COUNT(last_name) AS null_last_name,
    COUNT(*) - COUNT(email) AS null_email,
```

```

        COUNT(*) - COUNT(phone) AS null_phone
FROM customers;

-- For transactions table
SELECT
    'transactions' AS table_name,
    COUNT(*) - COUNT(transaction_id) AS null_transaction_id,
    COUNT(*) - COUNT(account_id) AS null_account_id,
    COUNT(*) - COUNT(customer_id) AS null_customer_id,
    COUNT(*) - COUNT(branch_id) AS null_branch_id,
    COUNT(*) - COUNT(transaction_type_id) AS null_transaction
_type_id,
    COUNT(*) - COUNT(amount) AS null_amount,
    COUNT(*) - COUNT(transaction_datetime) AS null_transactio
n_datetime,
    COUNT(*) - COUNT(is_fraud) AS null_is_fraud
FROM transactions;

-- For accounts table
SELECT
    'accounts' AS table_name,
    COUNT(*) - COUNT(account_id) AS null_account_id,
    COUNT(*) - COUNT(customer_id) AS null_customer_id,
    COUNT(*) - COUNT(account_type_id) AS null_account_type_i
d,
    COUNT(*) - COUNT(account_status_id) AS null_account_statu
s_id,
    COUNT(*) - COUNT(balance) AS null_balance
FROM accounts;

-- For loans table
SELECT
    'loans' AS table_name,
    COUNT(*) - COUNT(loan_id) AS null_loan_id,
    COUNT(*) - COUNT(customer_id) AS null_customer_id,
    COUNT(*) - COUNT(loan_type_id) AS null_loan_type_id,

```

```

COUNT(*) - COUNT(loan_status_id) AS null_loan_status_id,
COUNT(*) - COUNT(loan_amount) AS null_loan_amount,
COUNT(*) - COUNT(interest_rate) AS null_interest_rate,
COUNT(*) - COUNT(loan_date) AS null_loan_date
FROM loans;

```

2. Duplicate Detection in Transactions

```

-- Identify fully duplicate rows in transactions table
SELECT
    transaction_id,
    account_id,
    customer_id,
    branch_id,
    transaction_type_id,
    amount,
    transaction_datetime,
    is_fraud,
    COUNT(*) AS duplicate_count
FROM transactions
GROUP BY
    transaction_id,
    account_id,
    customer_id,
    branch_id,
    transaction_type_id,
    amount,
    transaction_datetime,
    is_fraud
HAVING COUNT(*) > 1;

-- If transaction_id is unique, find duplicates on all other
columns
SELECT
    account_id,
    customer_id,

```

```

    branch_id,
    transaction_type_id,
    amount,
    transaction_datetime,
    is_fraud,
    COUNT(*) AS duplicate_count,
    GROUP_CONCAT(transaction_id) AS duplicate_ids
FROM transactions
GROUP BY
    account_id,
    customer_id,
    branch_id,
    transaction_type_id,
    amount,
    transaction_datetime,
    is_fraud
HAVING COUNT(*) > 1;

```

3. Referential Integrity Check

```

-- Find transactions with invalid customer_id
SELECT
    t.transaction_id,
    t.customer_id,
    t.account_id,
    t.amount,
    t.transaction_datetime
FROM transactions t
LEFT JOIN customers c ON t.customer_id = c.customer_id
WHERE c.customer_id IS NULL;

-- Comprehensive referential integrity check for all relation
ships
SELECT
    'transactions->customers' AS relationship,
    COUNT(*) AS invalid_records

```

```

FROM transactions t
LEFT JOIN customers c ON t.customer_id = c.customer_id
WHERE c.customer_id IS NULL

UNION ALL

SELECT
    'transactions->accounts',
    COUNT(*)
FROM transactions t
LEFT JOIN accounts a ON t.account_id = a.account_id
WHERE a.account_id IS NULL

UNION ALL

SELECT
    'accounts->customers',
    COUNT(*)
FROM accounts a
LEFT JOIN customers c ON a.customer_id = c.customer_id
WHERE c.customer_id IS NULL

UNION ALL

SELECT
    'loans->customers',
    COUNT(*)
FROM loans l
LEFT JOIN customers c ON l.customer_id = c.customer_id
WHERE c.customer_id IS NULL;

```

4. Date Format Validation

```

-- Check for inconsistent date formats in transaction_datetime
-- Assumes standard format is 'YYYY-MM-DD HH:MI:SS'

```

```

SELECT
    transaction_id,
    transaction_datetime,
    CASE
        WHEN transaction_datetime REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}$'
        THEN 'Valid'
        ELSE 'Invalid'
    END AS date_format_check
FROM transactions
WHERE transaction_datetime NOT REGEXP '^[0-9]{4}-[0-9]{2}-[0-9]{2} [0-9]{2}:[0-9]{2}:[0-9]{2}$';

-- Alternative: check for different formats
SELECT
    transaction_id,
    transaction_datetime,
    'Inconsistent format' AS issue
FROM transactions
WHERE
    transaction_datetime NOT LIKE '____-__-__ __:__:__'
    OR LENGTH(transaction_datetime) != 19;

```

5. Future Date Anomalies

```

-- Flag transactions with future dates
-- Assuming current date is '2024-12-31' (adjust as needed)
SELECT
    transaction_id,
    customer_id,
    account_id,
    amount,
    transaction_datetime,
    'Future date anomaly' AS anomaly_type
FROM transactions
WHERE transaction_datetime > NOW();

```

```
-- Or compare to maximum date in dataset
WITH max_date AS (
    SELECT MAX(transaction_datetime) AS max_dt FROM transactions
)
SELECT
    t.transaction_id,
    t.transaction_datetime,
    'Future date relative to dataset' AS anomaly_type
FROM transactions t, max_date m
WHERE t.transaction_datetime > m.max_dt;
```

Section 2: Data Profiling & Exploration

6. Record Counts for All Tables

```
SELECT 'customers' AS table_name, COUNT(*) AS row_count FROM
customers
UNION ALL
SELECT 'accounts', COUNT(*) FROM accounts
UNION ALL
SELECT 'loans', COUNT(*) FROM loans
UNION ALL
SELECT 'transactions', COUNT(*) FROM transactions
UNION ALL
SELECT 'customer_types', COUNT(*) FROM customer_types
UNION ALL
SELECT 'account_types', COUNT(*) FROM account_types
UNION ALL
SELECT 'account_statuses', COUNT(*) FROM account_statuses
UNION ALL
SELECT 'loan_statuses', COUNT(*) FROM loan_statuses
UNION ALL
SELECT 'transaction_types', COUNT(*) FROM transaction_types
```

```

UNION ALL
SELECT 'branches', COUNT(*) FROM branches
UNION ALL
SELECT 'addresses', COUNT(*) FROM addresses
ORDER BY row_count DESC;

```

7. Customer Type Distribution

```

SELECT
    ct.type_name AS customer_type,
    COUNT(*) AS customer_count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM customer
s), 2) AS percentage
FROM customers c
JOIN customer_types ct ON c.customer_type_id = ct.customer_ty
pe_id
GROUP BY ct.type_name
ORDER BY customer_count DESC;

```

8. Account Status Overview

```

SELECT
    ast.status_name AS account_status,
    COUNT(*) AS account_count,
    ROUND(AVG(a.balance), 2) AS avg_balance,
    MIN(a.balance) AS min_balance,
    MAX(a.balance) AS max_balance,
    SUM(a.balance) AS total_balance
FROM accounts a
JOIN account_statuses ast ON a.account_status_id = ast.account
t_status_id
GROUP BY ast.status_name
ORDER BY account_count DESC;

```

9. Branch Activity Ranking

```

SELECT
    b.branch_id,
    b.branch_name,
    b.location,
    COUNT(t.transaction_id) AS transaction_count,
    ROUND(SUM(t.amount), 2) AS total_volume,
    ROUND(AVG(t.amount), 2) AS avg_transaction_amount,
    RANK() OVER (ORDER BY SUM(t.amount) DESC) AS volume_rank
FROM branches b
LEFT JOIN transactions t ON b.branch_id = t.branch_id
GROUP BY b.branch_id, b.branch_name, b.location
ORDER BY total_volume DESC;

```

10. Top Transaction Types

```

SELECT
    tt.type_name AS transaction_type,
    COUNT(*) AS transaction_count,
    ROUND(SUM(t.amount), 2) AS total_amount,
    ROUND(AVG(t.amount), 2) AS avg_amount,
    COUNT(CASE WHEN t.is_fraud = 1 THEN 1 END) AS fraud_count
FROM transactions t
JOIN transaction_types tt ON t.transaction_type_id = tt.transaction_type_id
GROUP BY tt.type_name
ORDER BY transaction_count DESC
LIMIT 3;

```

Section 3: Transactional & Behavioral Analysis

11. Top 10 Transactions with Customer and Branch Details

```

SELECT
    t.transaction_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,

```

```

    b.branch_name,
    b.location AS branch_location,
    tt.type_name AS transaction_type,
    t.amount,
    t.transaction_datetime,
    CASE WHEN t.is_fraud = 1 THEN 'Fraud' ELSE 'Legitimate' E
ND AS fraud_status
FROM transactions t
JOIN customers c ON t.customer_id = c.customer_id
JOIN branches b ON t.branch_id = b.branch_id
JOIN transaction_types tt ON t.transaction_type_id = tt.trans
action_type_id
ORDER BY t.amount DESC
LIMIT 10;

```

12. Monthly Transaction Summary

```

SELECT
    DATE_FORMAT(t.transaction_datetime, '%Y-%m') AS month,
    COUNT(*) AS transaction_count,
    ROUND(SUM(t.amount), 2) AS total_amount,
    ROUND(AVG(t.amount), 2) AS avg_amount,
    ROUND(MIN(t.amount), 2) AS min_amount,
    ROUND(MAX(t.amount), 2) AS max_amount,
    COUNT(CASE WHEN t.is_fraud = 1 THEN 1 END) AS fraud_coun
t,
    ROUND(COUNT(CASE WHEN t.is_fraud = 1 THEN 1 END) * 100.0
/ COUNT(*), 2) AS fraud_percentage
FROM transactions t
GROUP BY DATE_FORMAT(t.transaction_datetime, '%Y-%m')
ORDER BY month;

```

13. Customers with High Average Transaction Amount

```

WITH customer_avg AS (
    SELECT

```

```

        t.customer_id,
        CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
        COUNT(*) AS transaction_count,
        ROUND(AVG(t.amount), 2) AS avg_transaction_amount,
        ROUND(SUM(t.amount), 2) AS total_spent
    FROM transactions t
    JOIN customers c ON t.customer_id = c.customer_id
    GROUP BY t.customer_id, c.first_name, c.last_name
    HAVING COUNT(*) >= 5 -- Filter for customers with at least 5 transactions
)
SELECT *
FROM customer_avg
WHERE avg_transaction_amount > (
    SELECT AVG(amount) FROM transactions
)
ORDER BY avg_transaction_amount DESC;

```

14. Customers with High Daily Transaction Frequency

```

WITH daily_transactions AS (
    SELECT
        customer_id,
        DATE(transaction_datetime) AS transaction_date,
        COUNT(*) AS daily_count
    FROM transactions
    GROUP BY customer_id, DATE(transaction_datetime)
    HAVING COUNT(*) > 20
)
SELECT
    dt.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    dt.transaction_date,
    dt.daily_count,
    ROUND(SUM(t.amount), 2) AS total_daily_amount

```

```

FROM daily_transactions dt
JOIN customers c ON dt.customer_id = c.customer_id
JOIN transactions t ON dt.customer_id = t.customer_id
    AND dt.transaction_date = DATE(t.transaction_datetime)
GROUP BY dt.customer_id, dt.transaction_date, dt.daily_count,
c.first_name, c.last_name
ORDER BY dt.daily_count DESC;

```

15. Fraud Flag Analysis

```

SELECT
    COUNT(*) AS total_transactions,
    SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) AS fraud_count,
    SUM(CASE WHEN is_fraud = 0 THEN 1 ELSE 0 END) AS legitimate_count,
    ROUND(SUM(CASE WHEN is_fraud = 1 THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS fraud_percentage,
    ROUND(SUM(CASE WHEN is_fraud = 1 THEN amount ELSE 0 END), 2) AS total_fraud_amount,
    ROUND(SUM(CASE WHEN is_fraud = 1 THEN amount ELSE 0 END) * 100.0 / SUM(amount), 2) AS fraud_amount_percentage
FROM transactions;

```

Section 4: Loan Portfolio & Credit Risk

16. Loan Status Distribution

```

SELECT
    ls.status_name AS loan_status,
    COUNT(*) AS loan_count,
    ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM loans), 2) AS percentage,
    ROUND(AVG(l.loan_amount), 2) AS avg_loan_amount,
    ROUND(SUM(l.loan_amount), 2) AS total_loan_amount,

```

```

        ROUND(AVG(l.interest_rate), 2) AS avg_interest_rate
FROM loans l
JOIN loan_statuses ls ON l.loan_status_id = ls.loan_status_id
GROUP BY ls.status_name
ORDER BY loan_count DESC;

```

17. Default Rate by Loan Type

```

SELECT
    lt.type_name AS loan_type,
    COUNT(*) AS total_loans,
    SUM(CASE WHEN ls.status_name = 'Default' THEN 1 ELSE 0 EN
D) AS default_count,
    ROUND(SUM(CASE WHEN ls.status_name = 'Default' THEN 1 ELS
E 0 END) * 100.0 / COUNT(*), 2) AS default_rate,
    ROUND(AVG(l.loan_amount), 2) AS avg_loan_amount,
    ROUND(AVG(l.interest_rate), 2) AS avg_interest_rate
FROM loans l
JOIN loan_statuses ls ON l.loan_status_id = ls.loan_status_id
JOIN loan_types lt ON l.loan_type_id = lt.loan_type_id
GROUP BY lt.type_name
ORDER BY default_rate DESC;

```

18. Average Interest Rate by Customer Type

```

SELECT
    ct.type_name AS customer_type,
    COUNT(l.loan_id) AS loan_count,
    ROUND(AVG(l.interest_rate), 2) AS avg_interest_rate,
    ROUND(MIN(l.interest_rate), 2) AS min_interest_rate,
    ROUND(MAX(l.interest_rate), 2) AS max_interest_rate,
    ROUND(AVG(l.loan_amount), 2) AS avg_loan_amount
FROM loans l
JOIN customers c ON l.customer_id = c.customer_id
JOIN customer_types ct ON c.customer_type_id = ct.customer_ty
pe_id

```

```
GROUP BY ct.type_name
ORDER BY avg_interest_rate DESC;
```

19. Delinquent Loan Summary

```
SELECT
    l.loan_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    ls.status_name AS loan_status,
    l.loan_amount,
    l.interest_rate,
    l.loan_date,
    DATEDIFF(CURDATE(), l.loan_date) AS days_since_issued
FROM loans l
JOIN customers c ON l.customer_id = c.customer_id
JOIN loan_statuses ls ON l.loan_status_id = ls.loan_status_id
WHERE ls.status_name IN ('Default', 'Late')
ORDER BY l.loan_amount DESC;

-- Summary totals
SELECT
    ls.status_name,
    COUNT(*) AS loan_count,
    ROUND(SUM(l.loan_amount), 2) AS total_outstanding,
    ROUND(AVG(l.loan_amount), 2) AS avg_outstanding
FROM loans l
JOIN loan_statuses ls ON l.loan_status_id = ls.loan_status_id
WHERE ls.status_name IN ('Default', 'Late')
GROUP BY ls.status_name;
```

20. Customer Loan History

```
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    ct.type_name AS customer_type,
```

```

COUNT(l.loan_id) AS total_loans,
ROUND(SUM(l.loan_amount), 2) AS total_borrowed,
ROUND(SUM(CASE WHEN ls.status_name = 'Default' THEN l.loan_amount ELSE 0 END), 2) AS total_defaulted,
ROUND(AVG(l.interest_rate), 2) AS avg_interest_rate,
ROUND(AVG(l.loan_amount), 2) AS avg_loan_amount,
ROUND(SUM(CASE WHEN ls.status_name = 'Default' THEN l.loan_amount ELSE 0 END) * 100.0 / NULLIF(SUM(l.loan_amount), 0), 2) AS default_rate
FROM customers c
LEFT JOIN loans l ON c.customer_id = l.customer_id
LEFT JOIN customer_types ct ON c.customer_type_id = ct.customer_type_id
LEFT JOIN loan_statuses ls ON l.loan_status_id = ls.loan_status_id
GROUP BY c.customer_id, c.first_name, c.last_name, ct.type_name
HAVING total_loans > 0
ORDER BY total_borrowed DESC;

```

Section 5: Complex Joins & Integrated Analysis

21. High-Risk, High-Balance Accounts

```

-- Find top 10% highest balance accounts
WITH high_balance_accounts AS (
    SELECT
        account_id,
        customer_id,
        balance,
        PERCENT_RANK() OVER (ORDER BY balance DESC) AS balance_percentile
    FROM accounts
    WHERE account_status_id = (SELECT account_status_id FROM account_statuses WHERE status_name = 'Active')

```

```

),
-- Find customers with defaulted loans
customers_with_default AS (
    SELECT DISTINCT customer_id
    FROM loans l
    JOIN loan_statuses ls ON l.loan_status_id = ls.loan_status_id
    WHERE ls.status_name = 'Default'
)
SELECT
    hba.account_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    hba.balance,
    COUNT(l.loan_id) AS total_loans,
    ROUND(SUM(l.loan_amount), 2) AS total_loan_amount
FROM high_balance_accounts hba
JOIN customers c ON hba.customer_id = c.customer_id
JOIN customers_with_default cwd ON hba.customer_id = cwd.customer_id
LEFT JOIN loans l ON c.customer_id = l.customer_id
WHERE hba.balance_percentile <= 0.10 -- Top 10%
GROUP BY hba.account_id, c.customer_id, c.first_name, c.last_name, hba.balance
ORDER BY hba.balance DESC;

```

22. Transaction & Loan Correlation

```

WITH customer_transaction_stats AS (
    SELECT
        customer_id,
        COUNT(*) AS transaction_count,
        ROUND(AVG(amount), 2) AS avg_transaction_amount,
        ROUND(SUM(amount), 2) AS total_transaction_amount
    FROM transactions
    GROUP BY customer_id
),

```

```

customer_loan_stats AS (
    SELECT
        customer_id,
        COUNT(*) AS loan_count,
        ROUND(SUM(loan_amount), 2) AS total_loan_balance,
        ROUND(AVG(interest_rate), 2) AS avg_interest_rate
    FROM loans
    GROUP BY customer_id
)
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    COALESCE(cts.transaction_count, 0) AS transaction_count,
    COALESCE(cts.avg_transaction_amount, 0) AS avg_transaction
n_amount,
    COALESCE(cts.total_transaction_amount, 0) AS total_transa
ction_amount,
    COALESCE(cls.loan_count, 0) AS loan_count,
    COALESCE(cls.total_loan_balance, 0) AS total_loan_balanc
e,
    COALESCE(cls.avg_interest_rate, 0) AS avg_interest_rate
FROM customers c
LEFT JOIN customer_transaction_stats cts ON c.customer_id = c
ts.customer_id
LEFT JOIN customer_loan_stats cls ON c.customer_id = cls.cust
omer_id
WHERE cts.customer_id IS NOT NULL AND cls.customer_id IS NOT
NULL
ORDER BY total_transaction_amount DESC
LIMIT 20;

```

23. Branch Performance & Risk

```

SELECT
    b.branch_id,
    b.branch_name,

```

```

b.location,
-- Transaction metrics
COUNT(DISTINCT t.transaction_id) AS transaction_count,
ROUND(SUM(t.amount), 2) AS total_transaction_volume,
ROUND(AVG(t.amount), 2) AS avg_transaction_amount,
COUNT(CASE WHEN t.is_fraud = 1 THEN 1 END) AS fraud_transactions,
-- Loan metrics for customers of this branch
COUNT(DISTINCT l.loan_id) AS loan_count,
ROUND(SUM(l.loan_amount), 2) AS total_loan_amount,
ROUND(AVG(l.loan_amount), 2) AS avg_loan_amount,
ROUND(SUM(CASE WHEN ls.status_name = 'Default' THEN l.loan_amount ELSE 0 END) * 100.0 / NULLIF(SUM(l.loan_amount), 0), 2) AS default_rate,
-- Combined risk score
ROUND((COUNT(CASE WHEN t.is_fraud = 1 THEN 1 END) * 1.0 / NULLIF(COUNT(t.transaction_id), 0)) * 100, 2) AS fraud_rate_percentage
FROM branches b
LEFT JOIN transactions t ON b.branch_id = t.branch_id
LEFT JOIN customers c ON t.customer_id = c.customer_id
LEFT JOIN loans l ON c.customer_id = l.customer_id
LEFT JOIN loan_statuses ls ON l.loan_status_id = ls.loan_status_id
GROUP BY b.branch_id, b.branch_name, b.location
ORDER BY total_transaction_volume DESC;

```

24. Customer Lifetime Value (Simplified)

```

WITH customer_transactions AS (
    SELECT
        customer_id,
        COUNT(*) AS transaction_count,
        ROUND(SUM(amount), 2) AS total_spent,
        ROUND(AVG(amount), 2) AS avg_spent
    FROM transactions

```

```

        GROUP BY customer_id
    ),
    customer_loans AS (
        SELECT
            customer_id,
            COUNT(*) AS loan_count,
            ROUND(SUM(loan_amount), 2) AS total_borrowed,
            ROUND(SUM(CASE WHEN ls.status_name = 'Default' THEN l
loan_amount ELSE 0 END), 2) AS total_defaulted
        FROM loans l
        JOIN loan_statuses ls ON l.loan_status_id = ls.loan_statu
s_id
        GROUP BY customer_id
    )
SELECT
    c.customer_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    ct.type_name AS customer_type,
    COALESCE(ctrx.transaction_count, 0) AS transaction_count,
    COALESCE(ctrx.total_spent, 0) AS total_spent,
    COALESCE(cloans.total_borrowed, 0) AS total_borrowed,
    COALESCE(cloans.total_defaulted, 0) AS total_defaulted,
    ROUND(COALESCE(ctrx.total_spent, 0) - COALESCE(cloans.tot
al_defaulted, 0), 2) AS customer_value
FROM customers c
LEFT JOIN customer_transactions ctrx ON c.customer_id = ctrx.
customer_id
LEFT JOIN customer_loans cloans ON c.customer_id = cloans.cus
tomer_id
LEFT JOIN customer_types ct ON c.customer_type_id = ct.custom
er_type_id
ORDER BY customer_value DESC
LIMIT 20;

```

25. Fraudulent Transaction Patterns

```

WITH fraud_stats AS (
    SELECT
        tt.type_name AS transaction_type,
        COUNT(*) AS total_count,
        SUM(CASE WHEN t.is_fraud = 1 THEN 1 ELSE 0 END) AS fraud_count,
        ROUND(AVG(CASE WHEN t.is_fraud = 1 THEN t.amount END), 2) AS avg_fraud_amount,
        ROUND(AVG(CASE WHEN t.is_fraud = 1 THEN 1 END) * 100.0 / COUNT(*), 2) AS fraud_rate,
        b.branch_name AS most_frequent_branch,
        ROUND(MIN(t.amount), 2) AS min_fraud_amount,
        ROUND(MAX(t.amount), 2) AS max_fraud_amount
    FROM transactions t
    JOIN transaction_types tt ON t.transaction_type_id = tt.transaction_type_id
    LEFT JOIN branches b ON t.branch_id = b.branch_id
    WHERE t.is_fraud = 1
    GROUP BY tt.type_name, b.branch_name
)
SELECT
    transaction_type,
    SUM(total_count) AS total_transactions,
    SUM(fraud_count) AS fraud_transactions,
    ROUND(SUM(fraud_count) * 100.0 / SUM(total_count), 2) AS overall_fraud_rate,
    ROUND(AVG(avg_fraud_amount), 2) AS avg_fraud_amount,
    MIN(min_fraud_amount) AS min_fraud_amount,
    MAX(max_fraud_amount) AS max_fraud_amount,
    GROUP_CONCAT(DISTINCT most_frequent_branch) AS affected_branches
FROM fraud_stats
GROUP BY transaction_type
ORDER BY fraud_transactions DESC;

```

Section 6: Advanced Analytics & Data Cleaning

26. Handling Missing Data (Imputation)

```
-- Impute missing address_id with placeholder
UPDATE customers
SET address_id = -1
WHERE address_id IS NULL;

-- Alternative: Impute with most common address for customer
type
WITH common_address AS (
    SELECT
        customer_type_id,
        address_id,
        COUNT(*) AS count,
        RANK() OVER (PARTITION BY customer_type_id ORDER BY C
COUNT(*) DESC) AS rank
    FROM customers
    WHERE address_id IS NOT NULL
    GROUP BY customer_type_id, address_id
)
UPDATE customers c
SET address_id = (
    SELECT address_id
    FROM common_address ca
    WHERE ca.customer_type_id = c.customer_type_id AND ca.ran
k = 1
)
WHERE c.address_id IS NULL;

-- Impute missing transaction amounts with median
WITH median_amount AS (
    SELECT PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY amoun
t) AS median
    FROM transactions
```

```

)
UPDATE transactions
SET amount = (SELECT median FROM median_amount)
WHERE amount IS NULL;

-- Check NULL counts after imputation
SELECT
    COUNT(*) - COUNT(address_id) AS null_address_id_count
FROM customers;

```

27. Standardizing Inconsistent Text

```

-- Find and standardize inconsistent customer type names
-- First, find variations
SELECT DISTINCT type_name AS original_value
FROM customer_types
WHERE type_name IS NOT NULL
ORDER BY type_name;

-- Example: Standardize 'Personal' vs 'personal' vs 'Person'
UPDATE customer_types
SET type_name = 'Personal'
WHERE LOWER(type_name) IN ('personal', 'person', 'individual');

UPDATE customer_types
SET type_name = 'Corporate'
WHERE LOWER(type_name) IN ('corporate', 'corporation', 'company', 'corp');

-- Similarly for branch names
SELECT DISTINCT branch_name, LENGTH(branch_name)
FROM branches
WHERE branch_name LIKE '% ' OR branch_name LIKE ' %';

-- Trim whitespace

```

```

UPDATE branches
SET branch_name = TRIM(branch_name);

-- Standardize location format (example)
UPDATE branches
SET location = UPPER(location);

-- Show standardization results
SELECT
    branch_name,
    location,
    CASE
        WHEN branch_name LIKE '%Main%' THEN 'Main Branch'
        WHEN branch_name LIKE '%Downtown%' THEN 'Downtown Branch'
        ELSE 'Other'
    END AS standardized_name
FROM branches;

```

28. Cohort Analysis (Customer Acquisition)

```

WITH first_transaction AS (
    SELECT
        customer_id,
        MIN(DATE(transaction_datetime)) AS first_transaction_date,
        DATE_FORMAT(MIN(transaction_datetime), '%Y-%m') AS cohort_month
    FROM transactions
    GROUP BY customer_id
),
cohort_analysis AS (
    SELECT
        ft.cohort_month,
        COUNT(DISTINCT ft.customer_id) AS new_customers,
        ROUND(AVG(t.amount), 2) AS avg_first_transaction,

```

```

        ROUND(SUM(CASE
            WHEN DATE(t.transaction_datetime) BETWEEN ft.firs
t_transaction_date
                AND DATE_ADD(ft.first_transaction_date, INTER
VAL 30 DAY)
            THEN t.amount
            ELSE 0
        END), 2) AS total_first_month_value
FROM first_transaction ft
JOIN transactions t ON ft.customer_id = t.customer_id
GROUP BY ft.cohort_month
)
SELECT
    cohort_month,
    new_customers,
    avg_first_transaction,
    total_first_month_value,
    ROUND(total_first_month_value / NULLIF(new_customers, 0),
2) AS avg_revenue_per_customer
FROM cohort_analysis
ORDER BY cohort_month;

```

29. Moving Average of Daily Transactions

```

WITH daily_summary AS (
    SELECT
        DATE(transaction_datetime) AS transaction_date,
        ROUND(SUM(amount), 2) AS daily_total,
        COUNT(*) AS daily_count
    FROM transactions
    GROUP BY DATE(transaction_datetime)
)
SELECT
    transaction_date,
    daily_total,
    daily_count,

```

```

ROUND(AVG(daily_total) OVER (
    ORDER BY transaction_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
), 2) AS seven_day_moving_avg,
ROUND(AVG(daily_total) OVER (
    ORDER BY transaction_date
    ROWS BETWEEN 29 PRECEDING AND CURRENT ROW
), 2) AS thirty_day_moving_avg,
-- Trend indicator
CASE
    WHEN daily_total > AVG(daily_total) OVER (
        ORDER BY transaction_date
        ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
    ) THEN 'Above Average'
    ELSE 'Below Average'
END AS trend_indicator
FROM daily_summary
ORDER BY transaction_date DESC
LIMIT 30;

```

30. Integrated Data Quality Report

```

WITH quality_metrics AS (
    SELECT
        'customers' AS table_name,
        COUNT(*) AS total_rows,
        SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END)
AS null_customer_id,
        SUM(CASE WHEN first_name IS NULL THEN 1 ELSE 0 END) AS
null_first_name,
        SUM(CASE WHEN last_name IS NULL THEN 1 ELSE 0 END) AS
null_last_name,
        SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END) AS nul
l_email,
        COUNT(DISTINCT customer_id) AS unique_ids,
        COUNT(*) - COUNT(DISTINCT customer_id) AS duplicate_i

```

```

ds,
    0 AS referential_integrity_issues
FROM customers

UNION ALL

SELECT
    'transactions',
    COUNT(*),
    SUM(CASE WHEN transaction_id IS NULL THEN 1 ELSE 0 EN
D),
    SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END),
    SUM(CASE WHEN account_id IS NULL THEN 1 ELSE 0 END),
    SUM(CASE WHEN amount IS NULL THEN 1 ELSE 0 END),
    COUNT(DISTINCT transaction_id),
    COUNT(*) - COUNT(DISTINCT transaction_id),
    -- Referential integrity check for transactions
    SUM(CASE
        WHEN t.customer_id IS NOT NULL AND c.customer_id
IS NULL
        THEN 1 ELSE 0
    END)
FROM transactions t
LEFT JOIN customers c ON t.customer_id = c.customer_id

UNION ALL

SELECT
    'accounts',
    COUNT(*),
    SUM(CASE WHEN account_id IS NULL THEN 1 ELSE 0 END),
    SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END),
    SUM(CASE WHEN balance IS NULL THEN 1 ELSE 0 END),
    SUM(CASE WHEN account_status_id IS NULL THEN 1 ELSE 0
END),
    COUNT(DISTINCT account_id),

```

```

        COUNT(*) - COUNT(DISTINCT account_id),
        0
    FROM accounts
)
SELECT
    table_name,
    total_rows,
    null_customer_id + null_first_name + null_last_name + null_email AS total_nulls,
    ROUND((null_customer_id + null_first_name + null_last_name + null_email) * 100.0 / NULLIF(total_rows, 0), 2) AS null_percentage,
    duplicate_ids,
    ROUND(duplicate_ids * 100.0 / NULLIF(total_rows, 0), 2) AS duplicate_percentage,
    referential_integrity_issues,
    ROUND(referential_integrity_issues * 100.0 / NULLIF(total_rows, 0), 2) AS ri_issue_percentage,
    -- Quality Score (0-100)
    GREATEST(0, 100 -
        (null_percentage * 0.5) -
        (duplicate_percentage * 0.3) -
        (ri_issue_percentage * 0.2)
    ) AS data_quality_score
FROM quality_metrics
ORDER BY data_quality_score;

```

Bonus: Advanced Fraud Detection Queries

B1. Suspicious Pattern: Multiple Transactions Just Below Threshold

```

-- Find accounts with multiple transactions just below reporting threshold
WITH transaction_patterns AS (
    SELECT

```

```

        account_id,
        customer_id,
        COUNT(*) AS transaction_count,
        AVG(amount) AS avg_amount,
        SUM(amount) AS total_amount,
        MIN(amount) AS min_amount,
        MAX(amount) AS max_amount
    FROM transactions
    WHERE amount BETWEEN 9990 AND 9999.99 -- Just below $10,
000 threshold
    GROUP BY account_id, customer_id
    HAVING COUNT(*) >= 3
)
SELECT
    tp.*,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    a.balance,
    CASE
        WHEN tp.transaction_count >= 5 THEN 'High Risk'
        WHEN tp.transaction_count >= 3 THEN 'Medium Risk'
        ELSE 'Low Risk'
    END AS risk_level
FROM transaction_patterns tp
JOIN customers c ON tp.customer_id = c.customer_id
JOIN accounts a ON tp.account_id = a.account_id
ORDER BY tp.transaction_count DESC;

```

B2. Unusual Timing: Late-Night Transactions

```

-- Flag late-night transactions that are unusually high
SELECT
    t.transaction_id,
    CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
    t.amount,
    t.transaction_datetime,
    HOUR(t.transaction_datetime) AS hour_of_day,

```

```

CASE
    WHEN DAYOFWEEK(t.transaction_datetime) IN (1, 7) THEN
'Weekend'
    ELSE 'Weekday'
END AS day_type,
a.balance AS account_balance,
-- Flag based on thresholds
CASE
    WHEN t.amount > 1000 AND HOUR(t.transaction_datetime)
BETWEEN 0 AND 4 THEN 'High Value Late Night'
    WHEN t.amount > 500 AND HOUR(t.transaction_datetime)
BETWEEN 0 AND 4 THEN 'Medium Value Late Night'
    ELSE 'Normal'
END AS anomaly_flag
FROM transactions t
JOIN customers c ON t.customer_id = c.customer_id
JOIN accounts a ON t.account_id = a.account_id
WHERE HOUR(t.transaction_datetime) BETWEEN 0 AND 4 -- Between
n midnight and 4 AM
    AND t.amount > 500 -- Arbitrary threshold
    AND DAYOFWEEK(t.transaction_datetime) IN (1, 7) -- Weeke
nd
ORDER BY t.amount DESC;

```

These SQL solutions provide a comprehensive foundation for analyzing the finance fraud and loans dataset. They cover data quality assessment, exploratory analysis, fraud detection, credit risk evaluation, and advanced analytics. The queries are designed to be production-ready and can be easily adapted to your specific schema or business requirements.